

Design Doc – Cartesi Chainlink Lib

1. Introduction & Overview

This document outlines the design for the **Cartesi-Chainlink Automation Library**, a JavaScript/TypeScript library aimed at seamlessly integrating Chainlink Automation services with Cartesi dApps. The primary goal is to empower developers to implement advanced on-chain automation and leverage Cartesi's off-chain computation capabilities with unprecedented ease.

Currently, integrating Chainlink services, particularly Automation, into Cartesi dApps requires significant bespoke effort and deep familiarity with both ecosystems. This library will abstract much of that complexity, providing developer-friendly tools, APIs, smart contract templates, and deployment scripts.

A key innovation is the library's approach to interacting with Chainlink services "solely through code." This involves a deep programmatic integration with Chainlink's on-chain infrastructure, going beyond typical SDK usage to offer a streamlined experience specifically tailored for Cartesi developers, potentially by abstracting complex interactions with the Chainlink Registry and other relevant contracts. This is a pioneering effort to enable such deep integration for Cartesi.

2. Goals & Objectives

- **Seamless Integration:** Provide a straightforward way to trigger and manage Chainlink Automations that interact with Cartesi dApps.
- **Developer Empowerment:** Enable developers to implement sophisticated automation logic (time-based, custom logic, log-triggered) for Cartesi dApps without extensive manual setup via Chainlink UIs.
- **Unlock New Use Cases:** Facilitate advanced applications such as automated daily rewards in games, verifiable DeFi liquidations using off-chain models, and scalable automated lottery payouts within the Cartesi ecosystem.
- **Reduce Complexity:** Abstract the underlying intricacies of both Chainlink Automation and Cartesi input/output mechanisms.
- **Modularity & Extensibility:** Build a foundational architecture that can be extended to support other Chainlink services (e.g., Data Feeds, VRF, CCIP) in the future.
- **Enhance Cartesi Ecosystem:** Drive adoption and innovation by making powerful automation tools accessible to Cartesi developers.

3. Non-Goals

- Replicate the full Chainlink Automation UI.
- Manage user private keys directly (the library will expect a signer from a wallet provider like ethers.js).

- Provide a complete, standalone dApp (it's a library for developers to use in their dApps).
- Implement the core Cartesi Rollup or Chainlink Automation node software.
- Guarantee the economic viability or security of the specific dApp logic built using the library (developers are responsible for their dApp's security and tokenomics).
- Provide this lib in the multiple languages supported by the community

4. Target Audience

- Developers building dApps on the Cartesi platform.
- Developers familiar with JavaScript/TypeScript and smart contract development (Solidity).
- Projects looking to automate interactions with or within their Cartesi dApps using reliable, decentralized triggers.

5. Functional Requirements

FR1: Chainlink Automation Upkeep Registration

- **FR1.1:** The library MUST allow users to programmatically register Time-Based (cron) Upkeeps with Chainlink Automation, specifying all necessary parameters (target contract, cron schedule, gas limit, LINK funding, admin, `checkData`, `offchainConfig`).
- **FR1.2:** The library MUST allow users to programmatically register Custom Logic Upkeeps, specifying all necessary parameters (target, gas limit, LINK funding, admin, `checkData`, `offchainConfig`).
- **FR1.3:** The library MUST allow users to programmatically register Log-Triggered Upkeeps, specifying all necessary parameters (target, log emitter address, event signature/topics, gas limit, LINK funding, admin, `offchainConfig`).

FR2: Chainlink Automation Upkeep Management

- **FR2.1:** The library MUST allow users to programmatically retrieve detailed information about a registered Upkeep using its ID (e.g., configuration, LINK balance, status, admin).
- **FR2.2:** The library MUST allow users to programmatically add LINK funds to an existing Upkeep.
- **FR2.3:** The library MUST allow users to programmatically withdraw surplus LINK funds from an Upkeep (where Chainlink's system permits). - **need to check**
- **FR2.4:** The library MUST allow users to programmatically pause an active Upkeep.
- **FR2.5:** The library MUST allow users to programmatically unpause a paused Upkeep.
- **FR2.6:** The library MUST allow users to programmatically cancel an Upkeep.
- **FR2.7:** The library MUST allow users to programmatically update mutable parameters of an Upkeep (e.g., `checkData`, `gasLimit`, `offchainConfig`), as permitted by Chainlink contracts.

FR3: Cartesi dApp Interaction

- **FR3.1:** The library MUST provide utility functions to correctly format JavaScript data types (strings, numbers, objects) into a **bytes** payload suitable for Cartesi's **InputBox.addInput()** method.
- **FR3.2:** The library MUST support ABI-encoding of structured data or function calls into the **bytes** payload for Cartesi inputs if the Cartesi dApp expects such formats.
- **FR3.3:** The library MUST allow users to programmatically send a prepared input payload to a specified Cartesi dApp address via the **InputBox** contract, handling the on-chain transaction.

FR4: Smart Contract Support

- **FR4.1:** The library MUST provide at least one audited Solidity smart contract template that implements Chainlink's **AutomationCompatibleInterface** and is designed to forward calls to Cartesi's **InputBox**.
- **FR4.2:** The library MUST allow users to programmatically deploy this provided smart contract template to an EVM-compatible network.

FR5: Network and Configuration Management

- **FR5.1:** The library MUST allow users to configure it to interact with different EVM-compatible networks (e.g., Ethereum mainnet, Sepolia, Polygon, Mumbai).
- **FR5.2:** The library MUST provide default contract addresses (e.g., Chainlink Registry, LINK token, Cartesi **InputBox**) for commonly supported networks.
- **FR5.3:** The library MUST allow users to override default contract addresses or specify addresses for custom/unsupported networks.
- **FR5.4:** The library MUST integrate with standard Web3 providers and signers (e.g., from ethers.js or viem).

6. Non-Functional Requirements

NFR1: Usability / Developer Experience (DX)

- **NFR1.1:** The API MUST be well-documented, intuitive, and consistent.
- **NFR1.2:** The library MUST be easy to install via npm (or a similar package manager).
- **NFR1.3:** Error messages generated by the library MUST be clear, descriptive, and help developers diagnose issues.
- **NFR1.4:** The library MUST provide practical code examples and tutorials for common use cases.

NFR2: Reliability

- **NFR2.1:** The library's interactions with Chainlink and Cartesi smart contracts MUST be accurate and correct according to their specifications.

- **NFR2.2:** The library **MUST** behave predictably under valid and invalid input conditions.
- **NFR2.3:** The library **MUST** implement robust error handling for external calls (e.g., RPC requests, contract interactions).

NFR3: Security

- **NFR3.1:** The library's code **MUST NOT** introduce security vulnerabilities into the dApps that use it.
- **NFR3.2:** Any provided smart contract templates **MUST** be audited and follow security best practices (e.g., protection against reentrancy, proper access control).
- **NFR3.3:** The library **MUST** handle user-provided parameters securely to prevent transaction manipulation or unintended behavior.

NFR4: Performance

- **NFR4.1:** Client-side operations performed by the library (e.g., data formatting, transaction preparation) **MUST** execute efficiently and without undue delay.
- **NFR4.2:** The library **SHOULD NOT** introduce unnecessary on-chain transactions or excessive gas costs beyond what is inherent in interacting with Chainlink and Cartesi contracts.

NFR5: Maintainability

- **NFR5.1:** The codebase **MUST** be modular, well-structured, and follow good programming practices.
- **NFR5.2:** The code **MUST** be readable and adequately commented where complex logic exists.
- **NFR5.3:** The library **MUST** have comprehensive test coverage (unit, integration) to facilitate safe modifications and upgrades.

NFR6: Compatibility

- **NFR6.1:** The library **MUST** be compatible with current Long-Term Support (LTS) versions of Node.js.
- **NFR6.2:** The library **MUST** be compatible with widely-used Web3 libraries (e.g., ethers.js v6+, viem).
- **NFR6.3:** The library (if intended for frontend use) **MUST** be compatible with modern web browsers.

NFR7: Extensibility

- **NFR7.1:** The library's architecture **SHOULD** allow for future integration of other Chainlink services (e.g., Data Feeds, VRF, CCIP) with reasonable effort.

NFR8: Documentation

- **NFR8.1:** All public APIs, functions, classes, and types MUST be thoroughly documented.
- **NFR8.2:** Installation guides, usage tutorials, and examples MUST be provided and kept up-to-date.
- **NFR8.3:** Documentation MUST clearly explain any assumptions, limitations, or security considerations related to the library.

NFR9: Open Source

- **NFR9.1:** The library MUST be released under a permissive open-source license (e.g., MIT, Apache 2.0), as stated in the grant proposal.

7. PACELC Analysis - still building

It states that in case of a network **P**artition, a distributed system must choose between **A**vailability and **C**onsistency. **E**lse (during normal operation, without partitions), the system must choose between **L**atency and **C**onsistency. In our case, our lib is not a distributed system itself, but interacts with:

1. The Blockchain (e.g., Ethereum L1 or an L2) where Chainlink and Cartesi smart contracts are deployed.
2. The Chainlink Automation Network (decentralized off-chain nodes).
3. Cartesi Nodes (which run the off-chain Cartesi Machine and interact with the blockchain).

So, the PACELC analysis applies more to how these underlying systems behave and how the library will experience and interact with their trade-offs.

8. Technical Stack & Dependencies

- **Language:** TypeScript (compiled to JavaScript).
- **Blockchain Interaction:** `ethers.js` (v6.x) or `viem`.
- **Build Tool:** TSC, potentially Webpack/Rollup for bundling.
- **Testing:** Jest, Hardhat/Foundry (for local node and contract testing).
- **Distribution:** NPM.
- **Documentation:** TypeDoc, Docusaurus, or similar.

9. Security Considerations

- **Smart Contract Security:** All provided Solidity templates must be audited and follow best practices (reentrancy guards, access control, etc.).
- **Upkeep Management:** The library functions for registering/managing Upkeeps must ensure parameters are correctly encoded and validated to prevent misconfiguration.

- **performData Integrity:** Ensure that `performData` used in `performUpkeep` cannot be maliciously manipulated if it's derived from `checkData` or other sources.
- **Gas Limits:** Developers must be guided on setting appropriate gas limits for `performUpkeep`, especially as it includes a call to Cartesi's `InputBox`.
- **LINK Funding:** Clear guidance on maintaining sufficient LINK balance for Upkeeps.
- **Access Control:** Emphasize proper ownership and access control for deployed target contracts and Upkeep admin functions.
- **Input Sanitization (Cartesi side):** While not the library's direct responsibility, documentation should remind developers that Cartesi dApp backends must sanitize and validate all inputs received.
- **Programmatic Interaction Risks:** If interacting with Chainlink contracts in a non-standard way (beyond typical SDK usage), ensure these interactions are robust against potential changes in Chainlink's underlying contract interfaces or logic, if possible. Focus on stable, public interfaces.

10. Testing Strategy

- **Unit Tests:** For individual functions and classes within the library (mocking external dependencies).
- **Integration Tests:**
 - Testing library functions against a local Hardhat/Foundry node with deployed Chainlink mock contracts and Cartesi mock contracts (or actual core contracts if feasible).
 - Testing the end-to-end flow: Registering an Upkeep -> Triggering `performUpkeep` -> Verifying input added to a mock `InputBox`.
- **End-to-End (E2E) Tests:** (Post-MVP, on a public testnet)
 - Deploying a sample Cartesi dApp.
 - Using the library to set up Chainlink Automation on a testnet.
 - Verifying the complete cycle of trigger, Cartesi input, and expected Cartesi output (Notice/Voucher).
- **Code Coverage:** Aim for high code coverage.

11. Deployment & Distribution

- **NPM Package:** Published to the public NPM registry (`@prismatech/cartesi-chainlink-lib` or similar).
- **GitHub Repository:** Open-source, containing code, examples, and documentation.
- **Versioning:** Semantic Versioning (SemVer).

12. Documentation

- **API Reference:** Auto-generated from TSDoc comments.
- **Guides & Tutorials:**
 - Getting Started.
 - Step-by-step guides for each primary use case (daily rewards, DeFi liquidations, etc.).
 - Advanced configuration and troubleshooting.

- **Examples:** Fully functional example projects or snippets.
- **Security Best Practices:** Section on secure usage of the library.

13. Future Considerations & Roadmap

- **Support for other Chainlink Services:**
 - Chainlink Data Feeds (consuming feeds within Cartesi dApps).
 - Chainlink VRF (Verifiable Random Function for randomness in Cartesi dApps).
 - Chainlink CCIP (Cross-Chain Interoperability Protocol).
- **Enhanced `checkUpkeep` helpers:** Utilities or patterns for more complex `checkUpkeep` logic that might involve off-chain state from Cartesi Reports.
- **Gas Estimation Utilities.**
- **Support for different Cartesi compute instances/configurations.**
- **Community feedback incorporation.**

14. Open Questions & Risks

- **Complexity of "Solely Through Code" Chainlink Interaction:** What are the precise low-level interactions required for robust programmatic Upkeep registration and management beyond standard SDKs? How stable are these if they aren't part of public, versioned interfaces?
 - *Mitigation:* Prioritize using stable, documented Chainlink contract interfaces. If custom sequences are needed, thoroughly test and document assumptions.
- **User Adoption:** How to best encourage adoption and gather feedback from the Cartesi developer community?
 - *Mitigation:* Excellent documentation, examples, active community engagement, workshops.
- **Maintenance & Upgrades:** Keeping the library compatible with updates from both Chainlink and Cartesi.
 - *Mitigation:* Design for modularity, have a clear maintenance plan, and stay engaged with both communities.
- **Gas Cost Optimization:** Ensuring interactions are as gas-efficient as possible.
 - *Mitigation:* Diligent contract design and testing.
- **Security of Programmatic Registration:** Ensuring the methods provided for upkeep registration are secure and don't expose users to new risks.

15. Milestones

Week 1: Due 30/05

- **Project Scaffolding & Core Dependencies / Design Doc finalized:**
 - Initialize npm package with TypeScript, ESLint, Prettier, and testing framework (e.g., Jest).
 - Set up basic project structure (folders for `src`, `test`, `examples`, `contracts`).
 - Integrate core dependencies like `ethers.js` (or `viem`).
 - Basic `README.md` and `LICENSE` file.

- **Initial design of core interfaces/types** for Chainlink Upkeep configurations and Cartesi interactions.
-

Week 2: Due 06/06

- **Time-Based Upkeep Registration & Basic Cartesi Input:**
 - Implement core logic for **programmatic time-based (cron) Upkeep registration** with Chainlink Automation.
 - Implement basic functionality to **prepare and send a simple input** to a specified Cartesi dApp via **InputBox**.
 - Write initial unit tests for these core functionalities.
 - **Internal PoC:** Demonstrate registering a time-based Upkeep that successfully sends a predefined input to a test Cartesi dApp on a local/test network.
-

Week 3: Due 13/06

- **Custom Logic Upkeep Registration & Basic Upkeep Info:**
 - Implement core logic for **programmatic custom logic Upkeep registration**.
 - Develop function to **retrieve basic information about a registered Upkeep** (e.g., config, balance) by its ID.
 - Expand unit tests.
 - **Internal PoC:** Demonstrate registering a custom logic Upkeep.
-

Week 4: Due 20/06

- **Log Trigger Upkeep Registration & Basic Upkeep Management:**
 - Implement core logic for **programmatic log-triggered Upkeep registration**.
 - Implement basic Upkeep management functions: **funding an Upkeep (add LINK) and pausing an Upkeep**.
 - Further expand unit and initial integration tests for all three trigger types.
-

Week 5: Due 27/06

- **Smart Contract Templates & Deployment Helper:**
 - Finalize and test the Solidity smart contract template (**AutomationTargetForCartesiInput.sol**) designed to receive Chainlink calls and forward inputs to Cartesi.
 - Implement a library function to **programmatically deploy this target contract**.
 - Documentation for using the contract template.

Week 6: Due 04/07

- **Utility Functions & Initial Documentation:**
 - Implement key utility functions:
 - `checkData/performData` encoding/decoding helpers.
 - Network configuration helpers (presets for common networks).
 - Cron expression validator.
 - **Draft initial API documentation** (e.g., TypeDoc generated stubs) and basic usage guides.
 - Expand integration tests to cover utility functions and contract deployment.
-

Week 7: Due 11/07

- **Comprehensive Testing, Full Documentation, Examples & Beta Candidate:**
 - Write comprehensive integration and end-to-end tests for key use cases on a testnet.
 - Complete **full API documentation, detailed usage guides, and practical example scripts/projects**.
 - Refine error handling and user feedback.
 - **Prepare a Beta release candidate** of the npm package.
 - Ensure all common npm package best practices are met (Changelog, Contribution guidelines).